Deepfake Audio Detection

Preprocessing, Feature Extraction & Model Architecture

---

1. Preprocessing Pipeline

All audio files are loaded and standardised before feature extraction. The pipeline ensures that every sample is in a consistent format so the model receives uniform input regardless of the original recording conditions.

1.1 Audio Loading & Resampling

Audio files are loaded at a fixed sample rate of **16,000 Hz** and converted to **mono** using Librosa. Resampling to 16 kHz normalises recordings that may have been captured at different rates, and mono conversion removes channel-dependent variance.

```
audio, _ = librosa.load(filepath, sr=16000, mono=True)
```

1.2 Fixed-Length Normalisation (4 seconds)

Every waveform is clipped or zero-padded to exactly **64,000 samples** (4 seconds × 16,000 Hz). This produces identically shaped tensors, which is required for batched CNN training.

```
MAX_LEN = 64000 audio = np.pad(audio, (0, max(0, MAX_LEN - len(audio))))[:MAX_LEN]
```

2. Feature Extraction — Mel Spectrogram

Raw waveforms are transformed into **Mel Spectrograms**, a 2-D time-frequency representation that mirrors the non-linear frequency perception of the human auditory system. Spectral artefacts introduced by text-to-speech or voice-conversion systems are clearly visible in this domain, making it an effective feature for deepfake detection.

2.1 Mel Spectrogram Computation

```
mel = librosa.feature.melspectrogram(y=audio, sr=16000, n_mels=64) mel =
librosa.power_to_db(mel, ref=np.max)
```

Parameter	Value	Purpose
Sample Rate	16,000 Hz	Standard rate for speech processing
n_mels	64	Number of Mel filter banks
Max Length	64,000 samples	Fixed 4-second window
Scale	Power → dB	Compresses dynamic range for stability

2.2 Normalisation

Each spectrogram is **z-score normalised** per sample — zero mean, unit standard deviation — before being passed to the model. A small epsilon (1e-9) prevents division by zero on silent recordings.

```
mel = (mel - mel.mean()) / (mel.std() + 1e-9)
```

2.3 Tensor Shaping

A channel dimension is appended so the array matches the **(height, width, channels)** format expected by Keras Conv2D layers. The final shape per sample is **(64, 126, 1)**.

```
X.append(mel.astype(np.float32)) # shape (64, 126) X = np.array(X)[..., np.newaxis]
# shape (N, 64, 126, 1)
```

3. Model Architecture — Convolutional Neural Network

A sequential **Convolutional Neural Network (CNN)** is used to classify audio as *Genuine* or *Deepfake*. CNNs are well-suited for spectrogram inputs because they can learn spatially-local frequency and temporal patterns through shared convolutional filters.

3.1 Network Layout

Layer	Config	Role
Input	(64, 126, 1)	Mel Spectrogram tensor
Conv2D (Block 1)	32 filters, 3×3, ReLU	Low-level edge & frequency features
BatchNormalization	—	Stabilise activations
MaxPooling2D	2×2	Spatial down-sampling
Dropout	0.2	Regularisation
Conv2D (Block 2)	64 filters, 3×3, ReLU	Mid-level tonal patterns
BatchNormalization	—	Stabilise activations
MaxPooling2D	2×2	Spatial down-sampling
Dropout	0.2	Regularisation
Conv2D (Block 3)	128 filters, 3×3, ReLU	High-level speech structures
BatchNormalization	—	Stabilise activations
MaxPooling2D	2×2	Spatial down-sampling
Dropout	0.2	Regularisation
Conv2D (Block 4)	256 filters, 3×3, ReLU	Abstract discriminative features
BatchNormalization	—	Stabilise activations
GlobalAvgPooling2D	—	Compact feature vector; reduces overfitting
Dense	512 units, ReLU	Non-linear classification head
Dropout	0.5	Strong regularisation before output
Dense (Output)	2 units, Softmax	Genuine / Deepfake probabilities

3.2 Compilation

```
model.compile( optimizer = Adam(learning_rate=1e-3), loss =
'sparse_categorical_crossentropy', metrics = ['accuracy'] )
```

3.3 Class-Weight Balancing

The FOR-Norm dataset is imbalanced. Scikit-Learn's **compute_class_weight('balanced')** calculates per-class weights that are passed to model.fit(class_weight=...). This penalises misclassifications on the minority class more heavily, improving per-class accuracy without resampling.

3.4 Training Callbacks

- **ModelCheckpoint** — saves the best weights (monitored by val_accuracy).
- **ReduceLROnPlateau** — halves the learning rate after 3 epochs of no improvement in val_accuracy.
- **EarlyStopping** — stops training after 7 epochs without improvement and restores the best weights.

4. Evaluation Strategy

After training, the best saved model is evaluated on a held-out test set. Probability scores are used to derive the **Equal Error Rate (EER)** threshold — the point where False Accept Rate equals False Reject Rate — which is then applied to produce binary predictions.

Metric	Target	Description
Accuracy	≥ 80 %	Overall correct classifications
F1 Score	≥ 0.80	Harmonic mean of precision & recall
EER	≤ 12 %	Equal Error Rate (lower is better)
Per-class Acc.	≥ 75 %	Separate accuracy for Fake and Real classes